

Test Mühendisliği: Yeterlilik, Etkinlik, Güvenilirlik

Ömer Faruk Yavuz

August 2026

Özet

Bir hatanın giderilmesi için test yazılması problemi bitirmez; ikinci dereceden bir problem kümesi açar. Yazılan testin beklediği sonucun doğru olduğu, testin yeterli olduğu, testin gerçek hataları yakaladığı ve testin zaman içinde güvenilir kaldığı ayrı ayrı gösterilmesi gereken şeylerdir. Bu soruların her biri yazılım mühendisliği içinde bağımsız araştırma alanlarına dönüşmüştür.

Beklenen sonucun nereden bilineceği sorusu oracle problemi olarak adlandırılır ve genel çözümü yoktur; ancak doğru cevabı bilmeyi gerektirmeyen üç pratik yaklaşım geliştirilmiştir: özellik tabanlı test, metamorfik test ve mevcut çıktının sabitlenmesi. Bu yaklaşımların ortak özelliği, mutlak doğruyu değil girdiler arasındaki ilişkiyi kontrol etmeleridir.

Test yeterliliğinin ölçülmesinde kapsam (*coverage*) yaygın olarak kullanılır, ancak kapsam yalnızca negatif gösterge olarak geçerlidir: düşük kapsam kesin bir boşluğa işaret eder, yüksek kapsam hiçbir şeye işaret etmez. Kodun çalıştırılması ile davranışın doğrulanması farklı şeylerdir. Testlerin gerçekten hata yakalayıp yakalamadığını ölçen yöntem mutasyon testidir, ancak bu yöntemin de eşdeğer mutant ve hesaplama maliyeti biçiminde iki yapısal sınırı vardır.

Test kümesi büyüdükçe kendisi bakım gerektiren bir sisteme dönüşür. Bu nedenle kod tabanına uygulanan her bakım kavramı test kümesine de uygulanır; buna hiçbir hata yakalamayan testlerin kümeden çıkarılması da dahildir.

Hata maliyetinin çok yüksek olduğu alanlarda uygulanan yöntemler – biçimsel doğrulama, yedeklilik, bölümlendirme, izlenebilirlik, standart uyumu, arıza enjeksiyonlu simülasyon – hatayı sıfırlamaz, olasılığını düşürür. Bu alanlardaki en pahalı arızaların incelenmesi ortak bir bulgu vermektedir: kod çoğunlukla kendi spesifikasyonuna uygundur ve hata spesifikasyonun kendisindedir. Doğrulama ile geçirme arasındaki bu ayrım, test yatırımının nereye yapılacağını belirleyen esas ayrımdır.

Giriş

Yazılım testi, bir programın belirli girdiler için beklenen davranışı gösterip göstermediğinin gözlemlenmesidir. Bu tanım ilk bakışta tek bir faaliyeti tarif eder görünmesine rağmen, uygulamada birbirinden bağımsız birçok mühendislik problemi içerir.

Problemlerin ortaya çıkışını gösteren zincir şudur. Bir hata bulunur ve giderilir. Hatanın tekrarını engellemek için bir test yazılır. Bu noktada testin kendisi hakkında yeni sorular doğar: testin beklediği sonuç gerçekten doğru mudur, test kümesi yeterli midir, test gerçekten yanlış davranışı yakalayabilmekte midir, test kod değiştiğinde geçerliliğini koruyacak mıdır. Bu sorular birinci dereceden problemin – hatanın giderilmesinin – çözülmesiyle ortaya çıktığı için ikinci dereceden problemler olarak adlandırılabilir.

Notta düzeltilen yaygın kabul, kapsam ölçütünün test yeterliliğinin göstergesi sayılmasıdır. Kapsam, kodun hangi bölümlerinin çalıştırıldığını ölçer; davranışın doğrulanıp doğrulanmadığını ölçmez. Hiçbir doğrulama ifadesi (*assertion*) içermeyen bir test de tam kapsam üretebilir. Bu nedenle kapsamın ne söylediği ve ne söylemediği ayrı ayrı belirtilmelidir.

İkinci olarak, test faaliyetlerinin doğrusal bir sıra oluşturduğu kabulü de düzeltilmektedir. Bu faaliyetlerin bir kısmı gerçekten sıralıdır, bir kısmı test kümesine herhangi bir anda sorulabilecek kalite sorularıdır, bir kısmı ise bu soruların cevaplanmasında kullanılan ölçüm araçlarıdır. Üçünün aynı ekseninde gösterilmesi, aralarındaki mantıksal ilişkiyi gizler.

Merkezî bir terim ayrımı burada tanımlanmalıdır. Doğrulama (*verification*), sistemin kendi spesifikasyonuna uygun yapılp yapılmadığının kontrolüdür: ürün doğru mu yapıldı. Geçerleme (*validation*) ise spesifikasyonun kendisinin doğru olup olmadığının kontrolüdür: doğru ürün mü yapıldı. Notun son bölümünde görüleceği gibi, en pahalı yazılım arızalarının çoğu doğrulama değil geçerleme başarısızlığıdır.

Notun kapsamı test faaliyetinin yapısı, testin güvenilirliğinin ölçülmesi ve yüksek kritiklikteki alanlarda uygulanan yöntemlerle sınırlıdır. Belirli araçlar, kütüphaneler ve programlama dillerine özgü uygulamalar kapsam dışındadır.

Test Faaliyetinin Yapısı

Test işi tek bir eksende sıralanamaz. Birbirinden farklı üç eksen bulunur ve bunların karıştırılması, aralarındaki ilişkiyi görünmez kılar.

Birinci eksen: seviye

Testler, sistemin hangi ölçekte sındığına göre ayrılır. Her seviye farklı bir hata sınıfını yakalar ve maliyeti farklıdır.

- **Birim testi:** tek bir işlev veya sınıfın izole olarak sınanması. En ucuz ve en hızlı seviyedir; yalnızca o birimin iç mantığındaki hataları yakalar.
- **Entegrasyon testi:** birden fazla bileşenin birlikte çalışmasının sınanması. Bileşenlerin ayrı ayrı doğru, birlikte yanlış olduğu durumları yakalar.
- **Sözleşme testi** (*contract test*): iki taraf arasındaki arayüz beklentisinin her iki tarafta da aynı olduğunun sınanması. Servisler bağımsız olarak geliştirildiğinde, birinin sözleşmeyi sessizce değiştirmesini engeller.
- **Sistem testi:** bütünleşmiş sistemin gereksinimlerine karşı sınanması.
- **Uçtan uca test:** gerçek kullanıcı akışının baştan sona sınanması. En pahalı ve en yavaş seviyedir; buna karşılık başka hiçbir seviyenin göremediği bütünsel hataları yakalar.

Ayrıca doğru sonuç üretmeyi değil sistemin niteliklerini sınanan bir küme bulunur: performans, yük, dayanıklılık ve güvenlik testleri. Hata tanımı yalnızca “yanlış sonuç” ile sınırlandığında bu küme gözden kaçır; oysa bir sistemin doğru sonucu kabul edilemez bir sürede üretmesi de bir arızadır.

İkinci eksen: süreç adımları

Bunlar gerçekten sıralı faaliyetlerdir:

1. Test tasarımı: hangi durumların sınanacağını belirlenmesi.
2. Test durumu üretimi: sınanacak girdilerin oluşturulması.
3. Oracle belirlenmesi: beklenen sonucun ne olduğuna karar verilmesi.
4. Testin kodlanması.
5. Testin çalıştırılması.
6. Test bakımı: kod değişikçe testlerin güncel tutulması.

Test durumu üretimi maddesi bugün elle yazmakla sınırlı değildir. Rastgele ve bozuk girdi yağdırarak çökme arayan bulanıklaştırma (*fuzzing*) yöntemi, UNIX araçlarının rastgele girdilerle çökertilebildiğini gösteren bir çalışmayla ortaya çıkmış ve bugün girdi ayrıştırılan her bileşen için yerleşik bir yöntem haline gelmiştir (Miller ve diğerleri, 1990). Arama tabanlı yöntemler ise belirli bir kapsam hedefini sağlayan test kümelerini otomatik üretir.

Üçüncü eksen: kalite soruları ve ölçüm araçları

Bunlar bir adım değil, test kümesine herhangi bir anda sorulabilecek sorulardır:

- Yeterlilik (*adequacy*): küme yeterli mi.
- Etkinlik (*effectiveness*): küme gerçek hataları yakalıyor mu.
- Güvenilirlik (*reliability*): kümenin sonuçlarına güvenilebilir mi.

Kapsam ölçütleri ve mutasyon testi bu soruların cevaplanmasında kullanılan araçlardır; sürecin bir aşaması değildirler. Bu ayrımın pratik sonucu şudur: mutasyon testi regresyon testinden önce gelen bir aşama değil, regresyon kümesinin ne kadar iş gördüğünü ölçen bir araçtır.

Buna bağlı olarak, hangi testlerin ne zaman koşturulacağını belirleyen faaliyetler de ayrı bir küme oluşturur: değişiklikten sonra hangi testlerin koşturulacağını belirleyen seçme, koşturma sırasını belirleyen önceliklendirme, ve tüm süreci makineye devreden otomasyon. Bunların altyapısı ise test ortamı ve test verisi yönetimidir.

Oracle Problemi

Bir testin iki parçası vardır: girdi ve beklenen sonuç. Girdi üretmek görece kolaydır. Beklenen sonucun ne olduğuna karar vermek ise genel olarak çözümü olmayan bir problemidir ve literatürde test oracle problemi olarak adlandırılır (Barr ve diğerleri, 2015).

Problemin özü şu örnekte görülür. Bir fiyat hesaplama işlevinin belirli bir girdi için 80 döndürmesi beklenmektedir. Testin doğruluğu, 80 değerinin gerçekten doğru olmasına bağlıdır. Ancak testi yazan kişi de aynı yanlış anlayışa sahip olabilir; bu durumda hem kod hem test aynı hatayı içerir ve test her koşulda başarılı olur. Test, kodun spesifikasyona uygunluğunu değil, kodun testi yazanın zihnindeki modele uygunluğunu ölçmüş olur.

Problemin genel bir çözümü yoktur, çünkü çözüm doğru cevabı bağımsız olarak bilmeyi gerektirir; bu bilgi mevcut olsaydı zaten programa ihtiyaç olmazdı. Buna karşılık, doğru cevabı bilmeyi gerektirmeyen üç pratik yaklaşım geliştirilmiştir.

Özellik tabanlı test

Tek bir girdi için beklenen değer yerine, tüm girdiler için geçerli olması gereken bir özellik tanımlanır; girdileri araç üretir. Yöntem QuickCheck aracıyla yaygınlaşmıştır (Claessen ve Hughes, 2000).

Örneğin bir sıkıştırma işlevi için beklenen çıktının ne olduğu bilinmese dahi şu özellik bilinir: sıkıştırılıp açılan veri, başlangıçtaki veriyle aynı olmalıdır. Doğru cevabı bilmek gerekmez, ilişkiyi bilmek yeterlidir. Araç binlerce girdi dener ve özelliği çürüten bir girdi bulduğunda onu en küçük haline indirger, böylece sebep görünür hale gelir.

Metamorfik test

İki farklı girdinin sonuçları arasındaki ilişkiden çıkarım yapılır (Chen ve diğerleri, 1998). Bir arama işlevinin belirli bir sorgu için döndüreceği sonuç kümesi bilinmese dahi, sorguya bir kısıt eklendiğinde sonuç sayısının artmaması gerektiği bilinir. Bu ilişkiye metamorfik bağıntı denir.

Aynı yapı çok kullanıcıli sistemlerde de kurulabilir. Bir işlemin bir kullanıcı için doğru sonuç ürettiği doğrulandıktan sonra, aynı işlemin ikinci bir kullanıcı için de aynı biçimde davranması gerektiği bir metamorfik bağıntıdır. Bu tür bir sınama, birinci kullanıcının verisinin ikinciye sızması veya belirli bir kullanıcıya özel bir varsayımın koda gömülmüş olması gibi hataları yakalar; bu hataların tek kullanıcıli bir testte görünmesi mümkün değildir.

Mevcut çıktının sabitlenmesi

Sistemin bugünkü çıktısı doğru kabul edilip kaydedilir; sonraki koşullarda çıktı bu kayda karşı karşılaştırılır. Altın dosya (*golden file*) veya anlık görüntü testi

(*snapshot test*) olarak adlandırılır. Bakım bağlamındaki karşılığı karakterizasyon testidir; Feathers bu testi, davranışının belgelenmemiş olduğu devralınmış kodla çalışmanın başlangıç adımı olarak tanımlar (Feathers, 2004).

Yöntemin sınırı açıktır: çıktının doğru olduğunu göstermez, yalnızca değişmediğini gösterir. Mevcut davranış hatalıysa hata sabitlenmiş olur. Buna karşılık, hiçbir oracle bulunmayan durumda değişikliğin fark edilmesini sağlar.

Test Yeterliliğinin Ölçülmesi

Yeterlilik (*test adequacy*), bir test kümesinin yeterli sayılıp sayılamayacağı sorusudur. Zhu ve arkadaşlarının derlemesi bu alandaki ölçütleri sistematik olarak sınıflandırmıştır (Zhu ve diğerleri, 1997). Uygulamada en yaygın kullanılan ölçüt kapsamdır.

Kapsam ölçütleri

Kapsam, testler koşarken kodun hangi bölümlerinin çalıştırıldığını ölçer. Ölçütler artan güç sırasıyla şöyledir:

- **İfade kapsamı:** her satırın en az bir kez çalıştırılması.
- **Dal kapsamı:** her koşulun hem doğru hem yanlış dalının çalıştırılması.
- **Koşul kapsamı:** bileşik bir koşuldaki her alt koşulun her iki değeri de alması.
- **MC/DC** (*modified condition/decision coverage*): her alt koşulun, tek başına değiştiğinde sonucu değiştirdiğinin gösterilmesi. Havacılık ve otomotiv gibi güvenlik kritik alanlarda en yüksek kritiklik seviyesi için zorunlu tutulan ölçüttür.
- **Yol kapsamı:** program içindeki her yürütme yolunun çalıştırılması.

Yol kapsamı pratikte uygulanabilir bir ölçüt değildir. Bir döngü içeren herhangi bir programda olası yol sayısı hızla ele alınamaz büyüklüklere ulaşır. Ölçütler listesinde diğerleriyle aynı düzeyde gösterilmesi yanıltıcıdır.

Kapsamın ne söylediği

Kapsamın yorumlanmasında yapılan hata, ölçütü pozitif gösterge olarak kullanmaktır. Doğru formülasyon asimetriktir:

Kapsam negatif gösterge olarak güvenilirdir, pozitif gösterge olarak değildir.

Düşük kapsam kesin bilgi verir: çalıştırılmamış kodda hata bulunup bulunmadığı bilinmemektedir. Yüksek kapsam ise hiçbir şey söylemez, çünkü kapsam kodun çalıştırılmasını ölçer, davranışın doğrulanmasını değil. Hiçbir doğrulama ifadesi içermeyen, yalnızca işlevleri çağıran bir test kümesi de tam kapsam üretir ve hiçbir hata yakalamaz.

Mutasyon testi

Kapsamın cevaplamadığı soruyu – testler yanlış davranışı yakalayabiliyor mu – doğrudan ele alan yöntemdir. Fikir, kaynak kodda kasıtlı ve küçük değişiklikler yapıp test kümesinin bunları yakalayıp yakalamadığını ölçmektir. Yöntemin

kökene test verisi seçimi üzerine yazılmış erken bir çalışmadır (DeMillo ve diğerleri, 1978); alanın gelişimi Jia ve Harman tarafından derlenmiştir (Jia ve Harman, 2011).

İşleyiş şöyledir. Bir karşılaştırma operatörü tersine çevrilir, bir koşul kaldırılır veya bir sabit değiştirilir; ortaya çıkan programa mutant denir. Test kümesi koşturulur. Bir test başarısız oluyorsa mutant öldürülmüş sayılır ve küme o hatayı yakalayabildiğini göstermiştir. Hiçbir test başarısız olmuyorsa mutant hayatta kalmıştır ve küme o hata sınıfına karşı kör demektir.

İki ölçüt arasındaki fark şu iki soruyla ifade edilir: kapsam “hangi kodu çalıştırdım” diye sorar, mutasyon testi “yanlış davranışı yakalayabiliyor muyum” diye sorar. İkincisi, hata önleme problemine birincisinden yapısal olarak daha yakındır.

Yöntemin iki maliyeti bulunur. Birincisi eşdeğer mutant problemidir: bazı değişiklikler programın anlamını hiç değiştirmez, dolayısıyla hiçbir test onları yakalayamaz ve hayatta kalmaları kümenin eksikliğine işaret etmez. Bir mutantın eşdeğer olup olmadığının belirlenmesi genel olarak karar verilemez bir problemdir ve elle ayıklama gerektirir. İkincisi hesaplama maliyetidir: her mutant için test kümesinin baştan koşturulması gerekir. Bu iki nedenle mutasyon testi her değişiklikte koşan bir sürekli tümleştirme adımı olarak değil, dönemsel bir denetim aracı olarak konumlanır.

Göstergelerin gerçeklikten uzaklığı

Etkinliğin ölçülmesinde göstergeler bir zincir oluşturur ve zincirde aşağı inildikçe gerçeklikle bağ zayıflar:

```
üretime ulaşan hata sayısı  <- doğrudan ölçüm
~
mutasyon skoru              <- vekil ölçü
~
kapsam yüzdesi              <- vekilin vekili
```

Doğrudan ölçüm, test kümesinden kaçarak kullanıma ulaşan hata sayısıdır. Yanına konabilecek ikinci doğrudan gösterge, bir düzeltmenin yeni hata doğurma oranıdır; bu oran kümenin koruma gücünü doğrudan ölçer. Kapsam yüzdesinin bir hedef haline getirilmesi, ölçülen şeyin ölçülmek istenen şeyden iki adım uzakta olması nedeniyle yanıltıcıdır.

Test Kümesinin Kendisi Bir Sistemdir

Test kodu büyüdükçe kendisi bakım gerektiren bir yapıya dönüşür. Kod değiştiğinde testler kırılır, kırılan testler güncellenir, güncelleme sırasında testin taşıdığı bilgi bozulabilir. Bu döngü, kod tabanına uygulanan her bakım kavramının test kümesine de uygulandığı anlamına gelir: test kümesi de eskir, teknik borç biriktirir ve yeniden düzenlenmesi gerekir.

Test kokuları

Test kodundaki tekrarlayan yapısal sorunlar test kokusu (*test smell*) olarak adlandırılır ve sınıflandırılmıştır (van Deursen ve diğerleri, 2001). Yaygın olanları arasında bir testte hangi doğrulamanın başarısız olduğunun anlaşılamaması, testin dış bir dosyaya veya kayıtlı veriye bağımlı olması, tek bir testin birbiriyle ilgisiz birçok davranışı birden sınaması ve test kodunun kendi içinde koşullu mantık barındırması bulunur. Sonuncusu özellikle sorunludur: mantık içeren test kodunun kendisi de hata barındırabilir ve o mantığı sınavan bir test yoktur.

Kararsız testler

Aynı kod üzerinde bazen başarılı bazen başarısız olan testler kararsız test (*flaky test*) olarak adlandırılır. Nedenleri arasında zamanlamaya bağımlılık, testler arası paylaşılan durum, çalıştırma sırasına bağımlılık ve dış servislere erişim bulunur (Luo ve diğerleri, 2014).

Bu testlerin zararı doğrudan verdikleri yanlış sonuçtan büyüktür. Bir küme içinde belirli bir oranı aştıklarında, ekip başarısız testleri araştırmak yerine yeniden koşturmayı alışkanlık haline getirir. Bu noktadan sonra gerçek bir bozulma da aynı davranışla karşılanır ve kümenin koruma işlevi fiilen ortadan kalkar.

Test ikizlerinin ürettiği yanlış güven

Test ikizi (*test double*), bir bileşenin test sırasında gerçek bağımlılıkları yerine kullanılan sahte karşılıklardır; taklit (*mock*), sahte (*fake*) ve saplama (*stub*) gibi türleri vardır. İşlevleri, testi yavaş veya öngörülemez bağımlılıklardan yalıtmaaktır.

Aşırı kullanımının ürettiği sorun şudur: test, gerçek bileşen yerine testi yazan kişinin o bileşenin nasıl davrandığına dair varsayımını sınar. Varsayım yanlışsa, sistem bozuk olduğu halde test yeşil kalır. Bu, sistemin çalıştığına dair kanıt üretmeyen ama üretiyormuş gibi görünen bir yapı oluşturur ve testlerin verdiği güvenin en yaygın kırılma noktalarından biridir.

Test silmek bir bakım faaliyetidir

Yukarıdakilerin ortak sonucu, test kümesinin büyümesinin kendiliğinden bir kazanım olmadığıdır. Hiçbir hata yakalamayan, yalnızca kırıldığı için bakılan

testler kümeye maliyet ekler ve bilgi eklemey. Uygulama ayrıntısına bağlanmış testler ayrıca zarar verir: her yapısal düzenlemede kırıldıkları için yeniden düzenlemeyi caydırır ve tasarımın iyileştirilmesini engellerler.

Dolayısıyla test silmek de bir bakım faaliyetidir ve kümenin sağlığı için gereklidir. Ölçüt kümenin büyüklüğü değil, kümenin taşıdığı bilginin gerçekliğıdır.

Akla Gelmeyen Senaryo Problemi

Yukarıdaki tartışmaların hepsi, sınanacak durumun düşünülmüş olduğunu varsayar. Buna karşılık test kümesindeki en büyük boşluk çoğunlukla yanlış yazılmış testler değil, hiç yazılmamış testlerdir. Akla gelmeyen senaryo sınanamaz.

Bu bir keşif problemidir ve çözümü daha dikkatli düşünmek değil, keşfi insanın hayal gücüne bırakmayan yöntemler kullanmaktır. Dört yaklaşım bulunur ve dördü de farklı bir kaynaktan besleneceği için birbirini ikame etmez.

- **Rastgele girdi üretimi.** Özellik tabanlı testte insan yalnızca sağlanması gereken koşulu tanımlar, girdileri araç üretir. Sınır durumları insan sezgisinden sistematik olarak daha iyi bulunur, çünkü araç insanın ilginç bulmadığı bölgeleri de dolaşır.
- **Bulanıklaştırma.** Ne bekleneceği bilinmeden bozuk, aşırı ve anlamsız girdi üretilir; aranan şey doğru sonuç değil çökme, kilitlenme ve bellek ihlalidir. Dışarıdan veri alan her sınır için uygundur.
- **Mutasyon testi.** Bu bağlamda, kümenin hangi hata sınıflarına kör olduğunu gösterdiği için bir keşif aracı işlevi de görür.
- **Gerçek hatalardan geri besleme.** Kullanıma ulaşmış her hata için “bu neden testlerde görünmedi” sorusunun sorulması ve cevabın bir teste dönüştürülmesi. En verimli kaynak budur, çünkü hayal gücüne değil gerçekleşmiş olaya dayanır. Zamanla test kümesi, sistemin gerçek hata dağılımına göre şekillenir.

Dördüncü maddenin ayrıcalıklı konumu vurgulanmalıdır. İlk üçü olası hata uzayını tarar; dördüncüsü ise o uzayın hangi bölgesinin fiilen gerçekleştiğini bildirir. Bu bilgi başka hiçbir yöntemle üretilemez.

Hata Maliyetinin Yüksek Olduğu Alanlar

Havacılık, uzay, otomotiv ve tıbbi cihaz alanlarında bir yazılım hatasının maliyeti can kaybına kadar uzanabildiği için bu alanlarda çok daha ağır yöntemler uygulanır. Bu yöntemlerin incelenmesi iki bakımdan öğreticidir: hangi araçların mevcut olduğunu ve bu araçların uygulanmasına rağmen hataların neden tamamen ortadan kalkmadığını gösterir.

Uygulanan yöntemler

Biçimsel doğrulama

Kodun doğruluğunun denenerek değil kanıtlanarak gösterilmesidir. Test sonlu sayıda girdiyi dener; kanıt tüm girdiler için geçerlidir. Bunun için sistemin ne yapması gerektiği matematiksel bir dille yazılır – buna biçimsel spesifikasyon denir – ve kodun bu spesifikasyonu sağladığı araçlarla gösterilir. Başlıca yöntemler teorem ispatlayıcılar, sistemin tüm olası durumlarını dolaşan model denetimi ve değerlerin alabileceği aralıkları kodu çalıştırmadan hesaplayan soyut yorumlamadır.

Yöntemin sınırı belirleyicidir: kanıt, spesifikasyonun doğru olduğunu göstermez. Yanlış bir spesifikasyon kusursuz biçimde kanıtlanabilir. Ayrıca maliyeti yüksek olduğu için yalnızca en kritik çekirdek bileşenlere uygulanır.

Yedeklilik ve oylama

Aynı işi yapan birden fazla bağımsız birimin çalıştırılıp sonuçlarının karşılaştırılmasıdır. Üç birimde ikisinin uyduğu sonuç kabul edilir, ayrışan birim devre dışı bırakılır.

Donanım yedekliliği aynı yazılımı farklı bilgisayarlarda koşturur; donanım arızasına karşı korur, ancak yazılım hatası tüm kopyalarda aynı anda ortaya çıkar. Bu nedenle yazılım yedekliliği de uygulanır: aynı gereksinim birbirinden bağımsız ekipler tarafından, farklı yöntemlerle ayrı ayrı gerçekleştirilir.

Bu yaklaşımın dayandığı bağımsızlık varsayımı deneysel olarak sınanmış ve zayıf bulunmuştur. Knight ve Leveson, bağımsız olarak geliştirilmiş sürümlerin başarısızlıklarının istatistiksel olarak bağımsız olmadığını, farklı ekiplerin aynı zor noktalarda benzer hatalar yaptığını göstermiştir (Knight ve Leveson, 1986). Yöntem korelasyonu düşürür, ancak ortadan kaldırmaz.

Bölümleme

Kritik ve kritik olmayan yazılımın birbirini etkilemesinin engellenmesidir. Uçuş kontrolü ile yolcu eğlence sistemi aynı kaynakları paylaştığında, birinde ortaya çıkan bir hata veya aşırı kaynak tüketimi diğerini etkileyebilir.

Yöntemleri ayrı işlemciler, bellek koruma birimleri, her göreve sabit zaman dilimi ve sabit bellek ayıran gerçek zamanlı işletim sistemleri ve ayrılmış veri yollarıdır. Amaç, bir bileşenin arızasının komşusuna yayılmamasıdır.

İzlenebilirlik

Her gereksinimin hangi tasarım ögesine, hangi koda ve hangi teste karşılık geldiğinin uçtan uca kayıtlı olmasıdır. İlişki iki yönlü kurulur: bir gereksinimden onu karşılayan koda, bir kod parçasından onun hangi gereksinim için var olduğuna ulaşılabilir.

İki işlevi vardır. Birincisi, hiçbir gereksinimin karşılıksız kalmadığını gösterir. İkincisi, hiçbir gereksinime bağlanmayan kodun bulunmadığını gösterir; böyle bir kod kimsenin talep etmediği ve dolayısıyla sınanmamış koddur. Ayrıca bir değişikliğin etkilediği tüm öğelerin belirlenmesini sağlar; bu, değişiklik etki analizinin izlenebilirliğe dayanan biçimidir.

Standart uyumu

Bu alanlarda geliştirme süreci serbest değildir. Havacılıkta DO-178C, otomotivde ISO 26262, demiryolunda EN 50128 ve tıbbi cihazlarda IEC 62304 uygulanır.

Standartlar sisteme bir kritiklik seviyesi atar ve şartlar bu seviyeye göre ağırlaşır. En üst seviyede kaynak kodun MC/DC ölçütüne göre sınanması, her adımın belgelenmesi ve bağımsız gözden geçirme zorunludur. Uyum denetimle kanıtlanır; belgelenmemiş bir faaliyet yapılmamış sayılır.

Simülasyon ve arıza enjeksiyonu

Gerçek kullanımdan önce sistemin uzun süre benzetim ortamında çalıştırılmasıdır. Kademeleri tamamen yazılımsal benzetimden başlar, gerçek kontrol biriminin benzetilmiş sensörlere bağlanmasıyla sürer ve bütünlük yer testleriyle tamamlanır.

Bu aşamada aranan şey normal davranış değil arıza davranışdır: bir sensör bozulursa, iki sensör çelişirse, güç kesilirse veya veri yolu koparsa ne olacağı. Bu senaryolar gerçekte denenemeyeceği için benzetim ortamına kasıtlı olarak enjekte edilir.

Buna rağmen ortaya çıkan arızalar

Yukarıdaki yöntemlerin uygulandığı alanlarda dahi ciddi arızalar yaşanmıştır ve bunların incelenmesi ortak bir örüntü göstermektedir.

Ariane 5 roketinin ilk uçuşu kalkıştan kısa süre sonra kaybedilmiştir. İnceleme kurulunun raporuna göre nedeni, önceki modelden devralınan bir eylemsizlik referans sistemi yazılımında 64 bitlik bir kayan noktalı değerın 16 bitlik tam sayıya dönüştürülmesi sırasında oluşan taşmadır; ilgili modül yeni roketin uçuş profilinde işlevsel bir karşılığa sahip değildi (Lions, 1996). Kod, kendi spesifikasyonuna uygun çalışmaktaydı; hata, önceki roket için geçerli olan spesifikasyonun yeni roket için de geçerli sayılmasındaydı.

Mars Climate Orbiter aracı hedef yörüngeye giremeyerek kaybedilmiştir. İnceleme kurulu, yer yazılımının ürettiği itki verisinin bir birim sisteminde hesa-

planıp diğesinde yorumlanmasını temel neden olarak belirlemiştir (NASA, 1999). Her iki taraftaki yazılım da kendi içinde doğru çalışmaktaydı.

Boeing 737 MAX kazalarının incelemesinde, uçuş kontrol sisteminin tek bir hücum açısı sensörünün verisine dayanması ve bu sensörün hatalı veri üretmesi durumunda devreye giren davranışın belirleyici olduğu tespit edilmiştir. Yazılım tasarlandığı gibi çalışmaktaydı; sorun tasarım varsayımındaydı.

F-35 programında da yazılım kaynaklı sorunlar kamuya açık biçimde raporlanmıştır: radar sisteminin belirli koşullarda yeniden başlatma gerektirmesi, tarih çizgisi geçişinde navigasyon davranışının bozulması ve lojistik yazılımının sağlam birimleri arızalı olarak işaretlemesi bunlar arasındadır.

Ortak bulgu

Bu örneklerin ortak yanı, hataların çoğunda kodun kendi spesifikasyonuna uygun olmasıdır. Başarısızlık doğrulama düzeyinde değil geçişleme düzeyindedir: yanlış olan şey, kusursuz biçimde gerçekleştirilmiştir.

Bu bulgunun test yatırımı açısından sonucu şudur. Kod ile spesifikasyon arasındaki uyumu ölçen yöntemler – birim testi, kapsam, biçimsel kanıt – ne kadar güçlendirilirse güçlendirilsin, spesifikasyonun kendisindeki hatayı yakalayamaz. Bu hata sınıfını yakalayan şeyler farklı türdendir: varsayımların açıkça yazılması, arayüz sözleşmelerinde birim ve anlamın belirtilmesi, tek bir veri kaynağına dayanan kararların gözden geçirilmesi ve devralınan bileşenlerin yeni bağlamda hâlâ geçerli olup olmadığının sorgulanması.

Yöntemlerin tümünün dayandığı ilke, tek bir katmana güvenilmemesidir. Kanıt spesifikasyonu doğrulamaz, yedeklilik ortak nedenli hatayı yakalamaz, test her durumu göremez, standart uyumu belge düzeyinde kalabilir. Katmanlar üst üste konduğunda her birinin kör noktası bir diğeri tarafından kapatılır; bir hatanın zarara dönüşmesi için tüm katmanların aynı anda delinmesi gerekir. Bu yaklaşım güvenlik mühendisliğinde savunma katmanları olarak adlandırılır (Reason, 1997).

Sonuç

Test yazmak, hata önleme probleminin çözümü değil, o problemin ikinci bir düzeye taşınmasıdır. Yazılan her test, doğruluğu ayrıca gösterilmesi gereken bir iddiadır; kümenin bütünü ise yeterliliği, etkinliği ve güvenilirliği ayrı ayrı ölçülmesi gereken bir sistemdir. Bu nedenle “bug çıkmasın diye test yazalım” önerisi, kendi başına bir çözüm değil, çözülmesi gereken yeni bir problem kümesinin adıdır.

Bu kümede kullanılan ölçütlerin gerçeklikten uzaklığı hiyerarşiktir. Kullanıma ulaşan hata sayısı doğrudan ölçümdür; mutasyon skoru bunun vekilidir; kapsam yüzdesi vekilin vekilidir. Kapsamın hedef haline getirilmesi, ölçülmek istenen şeyden iki adım uzaktaki bir sayının optimize edilmesi anlamına gelir ve bu optimizasyonun sonucu, hata yakalamayan ama kapsam üreten testlerdir.

Oracle probleminin genel bir çözümü bulunmadığı için pratik yaklaşımlar mutlak doğruyu değil ilişkileri sınar. Bu, testin doğasına ilişkin bir kabulü zorunlu kılar: test, doğruluğu kanıtlamaz, belirli bir yanlışlık sınıfının bulunmadığını gösterir. Kanıt üreten tek yöntem biçimsel doğrulamadır ve o da yalnızca spesifikasyona göre kanıt üretir.

Çerçevenin en önemli sınırı bu son noktadadır. Yüksek maliyetli alanlarda yaşanan büyük arızaların incelenmesi, kodun genellikle spesifikasyonuna uygun olduğunu ve hatanın spesifikasyonda bulunduğunu göstermektedir. Doğrulama araçlarının hiçbirisi bu hata sınıfını yakalayamaz; bu, araçların yetersizliği değil, tanım gereği kapsamlarının dışında olmasıdır. Test yatırımının bir kısmının, kodun spesifikasyona uygunluğunu ölçen yöntemlerden, spesifikasyonun kendisini sorgulayan yöntemlere kaydırılması bu bulgunun doğrudan sonucudur.

Açık kalan soru, bir test kümesinin taşıdığı bilginin niceliksel olarak nasıl ifade edileceğidir. Mutasyon skoru bu bilginin bir kısmını ölçer ancak kümenin gereksiz büyümesini cezalandırmaz; kullanıma ulaşan hata sayısı gerçeği ölçer ancak geriye dönük ve yavaş bir göstergedir. İkisini birleştiren, kümenin hem koruma gücünü hem taşıdığı fazlalığı aynı anda değerlendiren bir ölçüt bulunmamaktadır.

Kaynaklar

- Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5).
- Chen, T. Y., Cheung, S. C., & Yiu, S. M. (1998). Metamorphic Testing: A New Approach for Generating Next Test Cases. Teknik rapor, Hong Kong University of Science and Technology.
- Claessen, K., & Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *ICFP*.
- DeMillo, R. A., Lipton, R. J., & Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*,

11(4).

- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Jia, Y., & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5).
- Knight, J. C., & Leveson, N. G. (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering*, SE-12(1).
- Leveson, N. G. (2011). *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press.
- Lions, J. L. (1996). *ARIANE 5 Flight 501 Failure: Report by the Inquiry Board*. European Space Agency.
- Luo, Q., Hariri, F., Eshkevari, L., & Marinov, D. (2014). An Empirical Analysis of Flaky Tests. *FSE*.
- Miller, B. P., Fredriksen, L., & So, B. (1990). An Empirical Study of the Reliability of UNIX Utilities. *Communications of the ACM*, 33(12).
- NASA (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*.
- Reason, J. (1997). *Managing the Risks of Organizational Accidents*. Ashgate.
- van Deursen, A., Moonen, L., van den Bergh, A., & Kok, G. (2001). Refactoring Test Code. *XP*.
- Zhu, H., Hall, P. A. V., & May, J. H. R. (1997). Software Unit Test Coverage and Adequacy. *ACM Computing Surveys*, 29(4).